

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Cuibari Vladislav FAF-243

Report

Laboratory Work No. 2

Operating Systems

Topic: Meet the OS You Will Build

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

CONTENTS

INTRODUCTION 3
1 THEORETICAL BACKGROUND 4
1.1 xv6 Architecture 4
1.2 System Calls and User Programs 4
2 PREPARE THE XV6 SOURCE TREE 5
2.1 Open or Clone the Repository 5
2.2 Check the Toolchain 5
3 BUILD AND BOOT XV6 7
3.1 Compile and Launch the System 7
4 USE XV6 AS A UNIX SYSTEM 8
4.1 Run Built-in Programs 8
4.2 Use a Pipe 8
5 READ THE XV6 SOURCE 10
5.1 Inspect User Programs 10
5.2 Locate System-call Interfaces 11
6 IMPLEMENT THE XV6 SLEEP PROGRAM 14
6.1 Create user/sleep.c 14
6.2 Register the Program in the Makefile 14
6.3 Rebuild and Test 15
CONCLUSION 17
REFERENCES 18

INTRODUCTION

This laboratory work introduces xv6, a compact Unix-like teaching operating system written in C for the RISC-V architecture. It covers the whole path from source code to a running system: obtaining the repository, cross-compiling the kernel and the user programs, booting the system in the QEMU emulator, using its shell, reading user-space and kernel-space code, and adding a new user program.

In Laboratory Work No. 1 Linux was inspected from the outside. xv6 is small enough to show both sides of an OS service: a user program issues a system call and a kernel handler performs the privileged work. The “Observe” questions of the handout are answered inside the comment of the matching exercise.

The work was done on Ubuntu under WSL2 (user userdalvgame) with the riscv64 cross-compiler GCC 15.2.0 and QEMU 10.2.1. Unlike the course VM, this WSL installation did not have the full toolchain, so it had to be completed first (section 2.2). Three problems appeared along the way (a missing QEMU package, a blank line in the Makefile, and a renamed system call); each one is described where it occurred, together with its fix. The sessions were recorded with the script command.

1 THEORETICAL BACKGROUND

1.1 xv6 Architecture

xv6 is a re-implementation of Unix Version 6 for a modern multiprocessor RISC-V machine. It consists of a privileged kernel that manages processes, memory, files, devices and system calls, and ordinary user programs that run with restricted privileges. It is normally executed inside QEMU, which emulates a RISC-V computer.

The source tree reflects this separation. The kernel/ directory holds the operating system itself: scheduler, virtual memory, file system, device drivers and system-call handlers. The user/ directory holds applications such as ls, cat, echo and the shell. The mkfs/ tool builds the disk image (fs.img) that contains these programs.

1.2 System Calls and User Programs

A user program cannot perform privileged operations itself. It calls functions declared in user/user.h; each one is a small assembly stub (generated from user/usys.pl) that executes the RISC-V ecall instruction and transfers control into the kernel. The kernel checks the request and runs the matching handler. For example, cat uses open, read, write and close, and the new sleep program asks the kernel to suspend it for a number of clock ticks.

2 PREPARE THE XV6 SOURCE TREE

2.1 Open or Clone the Repository

```
cd ~/xv6-riscv 2>/dev/null || \
  git clone https://github.com/mit-pdos/xv6-riscv ~/xv6-riscv
cd ~/xv6-riscv
ls
```

The evidence for this exercise is shown in Figure 2.1.

```
userdalvgame@WIN-07CNHC94OG2:~$ git clone https://github.com/mit-pdos/xv6-riscv.git ~/xv6-riscv
Cloning into '/home/userdalvgame/xv6-riscv'...
remote: Enumerating objects: 8334, done.
remote: Counting objects: 100% (33/33), done.
remote: Compressing objects: 100% (22/22), done.
remote: Total 8334 (delta 19), reused 11 (delta 11), pack-reused 8301 (from 2)
Receiving objects: 100% (8334/8334), 17.67 MiB | 512.00 KiB/s, done.
Resolving deltas: 100% (4583/4583), done.
$ cd ~/xv6-riscv
$ ls
LICENSE  Makefile  README  kernel  mkfs  test-xv6.py  user
```

Figure 2.1: The xv6 repository and its main source-tree entries

Comment and observation

The repository contains the kernel/ directory, the user/ directory, the mkfs/ tool, the Makefile, the README and the test script test-xv6.py. This confirms that the privileged operating-system code and the ordinary programs that run on top of it are in the same small source tree.

2.2 Check the Toolchain

```
git --version
make --version
riscv64-linux-gnu-gcc --version
qemu-system-riscv64 --version
# if qemu-system-riscv64 is missing:
sudo apt install -y qemu-system-riscv
```

The evidence for this exercise is shown in Figure 2.2.

```
$ git --version
git version 2.53.0
$ make --version | head -1
GNU Make 4.4.1
$ riscv64-linux-gnu-gcc --version | head -1
riscv64-linux-gnu-gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
$ qemu-system-riscv64 --version
Command 'qemu-system-riscv64' not found, but can be installed with:
sudo apt install qemu-system-riscv      # version 1:10.2.1+ds-1ubuntu3.1, or
sudo apt install qemu-system-riscv-hwe  # version 1:10.2.1+ds-1ubuntu4.2
$ sudo apt install -y qemu-system-riscv
$ command -v qemu-system-riscv64
/usr/bin/qemu-system-riscv64
$ qemu-system-riscv64 --version
QEMU emulator version 10.2.1 (Debian 1:10.2.1+ds-1ubuntu3.2)
```

Figure 2.2: Toolchain versions, the missing QEMU binary and its installation

Comment and observation

Git, GNU Make and the RISC-V cross-compiler were present, but the emulator command qemu-system-riscv64 was not found. I first installed the package qemu-system-misc, which the handout lists, and make qemu still failed; on this Ubuntu release the RISC-V emulator is shipped in the separate package qemu-system-riscv. After installing it, qemu-system-riscv64 resolved to /usr/bin/qemu-system-riscv64 and reported version 10.2.1. The lesson is to verify every tool with command -v or --version before the first build.

3 BUILD AND BOOT XV6

3.1 Compile and Launch the System

```
make qemu
```

The evidence for this exercise is shown in Figures 3.1–3.2.

```
$ make qemu       (first attempt, before installing qemu-system-riscv)
riscv64-linux-gnu-gcc ... -c -o kernel/entry.o kernel/entry.S
riscv64-linux-gnu-gcc ... -c -o kernel/start.o kernel/start.c
  ... (all kernel and user programs compile and link) ...
mkfs/mkfs fs.img README user/_cat user/_echo user/_forktest user/_grep ...
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
balloc: first 983 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic ...
make: qemu-system-riscv64: No such file or directory
make: *** [Makefile:183: qemu] Error 127
```

Figure 3.1: First attempt: the build succeeds but QEMU cannot be started

```
$ make qemu       (after installing qemu-system-riscv)
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic
  -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0
  -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0


xv6 kernel is booting


hart 2 starting
hart 1 starting
init: starting sh
$
```

Figure 3.2: Successful boot of xv6 under QEMU

Comment and observation

The first make qemu compiled every kernel file with riscv64-linux-gnu-gcc, linked them with kernel/kernel.ld into the kernel image, built the user programs, and let the host tool mkfs/mkfs pack them (with the README) into the filesystem image fs.img. It stopped only at the last step, because the emulator was missing (Error 127 = command not found); the "(standard_in) syntax error" lines come from the Makefile's QEMU version check, which received an empty version string for the same reason.

After qemu-system-riscv was installed, the same command started QEMU with a RISC-V "virt" machine, 128 MB of RAM and three CPUs (-smp 3), the kernel image and the disk image attached as a virtio block device. The messages xv6 kernel is booting, hart 1 starting and hart 2 starting (a hart is a hardware thread, i.e. a CPU core) and init: starting sh, followed by the $ prompt, show that the kernel initialised, started the other cores, and launched the first user-space process, which runs the shell.

Leaving QEMU: press Ctrl-A, release both keys, then press X (QEMU printed "QEMU: Terminated" when I did this).

4 USE XV6 AS A UNIX SYSTEM

4.1 Run Built-in Programs

```
$ ls
$ cat README
$ echo hello xv6
$ wc README
```

The evidence for this exercise is shown in Figure 4.1.

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
forktest       2 5 18136
grep           2 6 44184
init           2 7 35976
kill           2 8 35512
ln             2 9 35304
ls             2 10 42880
mkdir          2 11 35568
rm             2 12 35552
sh             2 13 58312
stressfs       2 14 36416
usertests      2 15 206800
grind          2 16 51824
wc             2 17 37728
zombie         2 18 34848
logstress      2 19 37560
forphan        2 20 36352
dorphan        2 21 35800
sync           2 22 34944
console        3 23 0
$ cat README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
but is implemented for a modern RISC-V multiprocessor using ANSI C.
  ... (rest of README omitted) ...
$ echo hello xv6
hello xv6
$ wc README
48 336 2441 README
```

Figure 4.1: Execution of built-in xv6 user programs

Comment and observation

xv6 ships with many small programs; three of them are ls, cat and echo (also grep, wc, mkdir, rm, ln, kill and the shell sh). The xv6 ls prints, for each entry, its name, its type (1 = directory, 2 = file, 3 = device), its inode number and its size in bytes, so it is simpler than the Linux ls. cat README printed the file, echo printed its argument, and wc README reported 48 lines, 336 words and 2441 bytes, which matches the size shown by ls.

4.2 Use a Pipe

```
$ ls | grep c
```

The evidence for this exercise is shown in Figure 4.2.

```
$ ls | grep c
cat            2 3 36728
echo           2 4 35592
wc             2 17 37728
sync           2 22 34944
console        3 23 0
```

Figure 4.2: A pipeline connecting ls to grep inside xv6

Comment and observation

The pipe works as in Linux: grep c keeps only the lines that contain the letter c. The result is exactly the entries whose names contain a c: cat, echo, wc, sync and console. Every other name in the earlier listing (grep, init, ls, sh, ...) has no c and was filtered out, so the output of ls was really consumed by grep.

For a pipe to work, the OS needs at least two features: process management and inter-process communication. The shell has to create two processes (fork and exec) for ls and grep, create a kernel-managed pipe (the pipe system call, a buffer in the kernel with a read end and a write end), connect the standard output of ls to the write end and the standard input of grep to the read end by duplicating file descriptors, and wait for both to finish. Compared with the Linux shell of Laboratory Work No. 1, the xv6 shell is intentionally minimal: it understands pipes, redirection and command lists, but it has no command history, tab completion, variables such as $PID or job control with jobs.

5 READ THE XV6 SOURCE

5.1 Inspect User Programs

QEMU was closed first (Ctrl-A, X) so that the source tree could be read from the Linux shell.

```
sed -n "1,40p" user/ls.c
sed -n "1,30p" user/cat.c
```

The evidence for this exercise is shown in Figures 5.1–5.2.

```
$ sed -n '1,40p' user/ls.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"


char *
fmtname(char *path)
{
  static char buf[DIRSIZ + 1];
  char *p;


  // Find first character after last slash.
  for (p = path + strlen(path); p >= path && *p != '/'; p--)
    ;
  p++;


  // Return blank-padded name.
  if (strlen(p) >= DIRSIZ)
    return p;
  memmove(buf, p, strlen(p));
  memset(buf + strlen(p), ' ', DIRSIZ - strlen(p));
  buf[sizeof(buf) - 1] = '\0';
  return buf;
}


void
ls(char *path)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;


  if ((fd = open(path, O_RDONLY)) < 0) {
    fprintf(2, "ls: cannot open %s\n", path);
    return;
  }


  if (fstat(fd, &st) < 0) {
```

Figure 5.1: Source code of user/ls.c (lines 1–40)

```
$ sed -n '1,80p' user/cat.c
#include "kernel/types.h"
#include "kernel/fcntl.h"
#include "user/user.h"

char buf[512];

void
cat(int fd)
{
  int n;

  while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
      fprintf(2, "cat: write error\n");
      exit(1);
    }
  }
  if (n < 0) {
    fprintf(2, "cat: read error\n");
    exit(1);
  }
}

int
main(int argc, char *argv[])
{
  int fd, i;

  if (argc <= 1) {
    cat(0);
    exit(0);
  }

  for (i = 1; i < argc; i++) {
    if ((fd = open(argv[i], O_RDONLY)) < 0) {
      fprintf(2, "cat: cannot open %s\n", argv[i]);
      exit(1);
    }
    cat(fd);
    close(fd);
  }
  exit(0);
}
```

Figure 5.2: Source code of user/cat.c (whole file, 43 lines)

Comment and observation

The source shows that xv6 user programs are small ordinary C programs: a main function plus calls to the user API. cat reads from each file given on the command line, or from standard input (descriptor 0) when no argument is given. It opens the file, repeatedly reads up to 512 bytes into a buffer, writes them to standard output (descriptor 1), closes the descriptor, and finally exits. Errors are reported on descriptor 2 with fprintf and the program exits with status 1.

5.2 Locate System-call Interfaces

```
cat user/user.h
grep -n "sys_read\|sys_write" kernel/sysfile.c | head
grep -n "open\|read\|write\|close\|exit" user/cat.c
```

The evidence for this exercise is shown in Figures 5.3–5.4.

```
$ cat user/user.h
#define SBRK_ERROR ((char *)-1)


struct stat;


// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int *);
int pipe(int *);
int write(int, const void *, int);
int read(int, void *, int);
int close(int);
int kill(int);
int exec(const char *, char **);
int open(const char *, int);
int mknod(const char *, short, short);
int unlink(const char *);
int fstat(int fd, struct stat *);
int link(const char *, const char *);
int mkdir(const char *);
int chdir(const char *);
int dup(int);
int getpid(void);
char *sys_sbrk(int, int);
int pause(int);
int uptime(void);
int sync(void);


// ulib.c
  ... (ulib, printf and umalloc declarations omitted) ...
```

Figure 5.3: User-visible declarations of the system calls (user/user.h)

```
$ grep -n "sys_read\|sys_write" kernel/sysfile.c | head
69:sys_read(void)
83:sys_write(void)


$ grep -n "open\|read\|write\|close\|exit" user/cat.c
12:  while ((n = read(fd, buf, sizeof(buf))) > 0) {
13:    if (write(1, buf, n) != n) {
14:      fprintf(2, "cat: write error\n");
15:      exit(1);
19:    fprintf(2, "cat: read error\n");
20:    exit(1);
31:    exit(0);
35:    if ((fd = open(argv[i], O_RDONLY)) < 0) {
36:      fprintf(2, "cat: cannot open %s\n", argv[i]);
37:      exit(1);
40:    close(fd);
42:  exit(0);
```

Figure 5.4: Kernel implementations of read and write, and the system calls used by cat

Comment and observation

user/cat.c uses five system calls. open(path, O_RDONLY) asks the kernel for a file descriptor for the named file; read(fd, buf, n) asks the kernel to copy up to n bytes of the file into the user buffer; write(1, buf, n) asks it to send those bytes to standard output; close(fd) releases the descriptor; and exit(status) terminates the process. (fprintf(2, …) ends in write on descriptor 2.)

The declarations are in user/user.h. sys_read is implemented in kernel/sysfile.c on line 69 of this checkout, and sys_write on line 83. The link between the two sides is the file user/usys.S, which the build generates from user/usys.pl (visible in the make output as “perl user/usys.pl >

user/usys.S”): every stub loads the system-call number and executes ecall, and the kernel then dispatches to the matching sys_… function.

The difference between kernel/ and user/: code in kernel/ is the operating system. It runs in privileged mode and manages the resources (processes, memory, files, devices). Code in user/ runs unprivileged and can reach those resources only by asking the kernel through system calls.

6 IMPLEMENT THE XV6 SLEEP PROGRAM

6.1 Create user/sleep.c

The program user/sleep.c was created with the following final content:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }

  pause(atoi(argv[1]));
  exit(0);
}
```

The evidence for this exercise is shown in Figure 6.1.

```
$ nano user/sleep.c
$ sed -n '1,80p' user/sleep.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }

  pause(atoi(argv[1]));
  exit(0);
}
```

Figure 6.1: Implementation of user/sleep.c

Comment and observation

The program checks that exactly one argument was given. If it was not, it prints the usage message on descriptor 2 (standard error) and exits with status 1. Otherwise it converts the text argument to an integer with atoi, calls the system call to pause for that many clock ticks, and exits with status 0.

Note on the system-call name: the handout calls sleep(…), but the version of xv6 that I cloned has renamed this system call to pause. user/user.h declares int pause(int); and contains no sleep for user programs (sleep remains an internal kernel function). My first build with sleep(…) failed for this reason (section 6.3), so the final program calls pause(…).

6.2 Register the Program in the Makefile

$U/_sleep must be added to the UPROGS list. The line has to start with a tab and end with a backslash.

```
UPROGS=\
        $U/_cat\
        $U/_echo\
        $U/_sleep\
        $U/_forktest\
        ...
```

The evidence for this exercise is shown in Figures 6.2–6.3.

```
$ make clean
Makefile:134: *** recipe commences before first target.  Stop.
$ nl -ba Makefile | sed -n '125,140p'     (after my first edit in nano)
   130  .PRECIOUS: %.o
   131
   132  UPROGS=\
   133           <-- accidental empty line
   134    $U/_cat\
   135    $U/_echo\
   136    $U/_sleep\
```

Figure 6.2: First attempt: a blank line inside UPROGS breaks the Makefile

```
$ sed -i '133d' Makefile
$ nl -ba Makefile | sed -n '130,140p'
   130  .PRECIOUS: %.o
   131
   132  UPROGS=\
   133    $U/_cat\
   134    $U/_echo\
   135    $U/_sleep\
   136    $U/_forktest\
   137    $U/_grep\
$ git status --short
 M Makefile
?? user/sleep.c
```

Figure 6.3: Makefile after removing the empty line, and the changed files

Comment and observation

Adding $U/_sleep to UPROGS tells the build to compile user/sleep.c into the executable user/_sleep and to include it in fs.img. Without this line the source file would exist on the host but the command would not exist inside xv6.

My first edit left an empty line directly after UPROGS=\. In a Makefile a blank line ends the variable definition, so the tab-indented lines that followed were read as a rule recipe, and make stopped with “recipe commences before first target”. Deleting line 133 (sed -i '133d' Makefile) fixed it. git status confirms that exactly two things changed: the modified Makefile and the new, untracked user/sleep.c.

6.3 Rebuild and Test

```
make clean
make qemu
$ sleep 10
$ sleep
```

The evidence for this exercise is shown in Figures 6.4–6.5.

```
$ make qemu       (with sleep(atoi(argv[1])) in user/sleep.c)
riscv64-linux-gnu-gcc ... -c -o user/sleep.o user/sleep.c
user/sleep.c: In function 'main':
user/sleep.c:12:3: error: implicit declaration of function 'sleep' [-Wimplicit-function-declaration]
   12 |   sleep(atoi(argv[1]));
      |   ^~~~~
make: *** [<builtin>: user/sleep.o] Error 1


$ sed -i 's/sleep(atoi(argv\[1\]));/pause(atoi(argv[1]));/' user/sleep.c
$ grep -n 'pause' user/user.h user/sleep.c
user/user.h:25:int pause(int);
user/sleep.c:12:  pause(atoi(argv[1]));
```

Figure 6.4: Compilation error caused by the renamed system call, and the fix

```
$ make qemu
riscv64-linux-gnu-ld ... -o user/_sleep user/sleep.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
mkfs/mkfs fs.img README user/_cat user/_echo user/_sleep user/_forktest ...
balloc: first 1019 blocks have been allocated
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic ...


xv6 kernel is booting


hart 2 starting
hart 1 starting
init: starting sh
$ ls | grep sleep
sleep          2 5 35280
$ sleep 10
$ sleep
usage: sleep <ticks>
$
```

Figure 6.5: Successful execution and argument validation of the xv6 sleep program

Comment and observation

With sleep(…) the compiler stopped with “implicit declaration of function ‘sleep’”: user programs only see the declarations in user/user.h, and that header has pause. Because the build uses -Wall -Werror, the warning became an error. Replacing the call with pause(…) (confirmed with grep: user/user.h line 25 and user/sleep.c line 12) made the build succeed: user/_sleep was linked, mkfs added it to fs.img, and xv6 booted.

Inside xv6, ls | grep sleep shows the new program (type 2, inode 5, 35280 bytes). Because it was inserted after echo in UPROGS, the inode numbers of the programs after it shifted by one compared with the earlier listing. sleep 10 waits about ten clock ticks (roughly a second in QEMU) and returns to the prompt, which shows that the program reaches the kernel’s sleep service. Running sleep without an argument prints usage: sleep <ticks>, which confirms that the argument-count check and the error path work.

CONCLUSION

The xv6 source was cloned, built with the RISC-V cross-compiler and booted under QEMU, which shows how a kernel, user programs and a filesystem image are combined into a working operating system. Fixing the toolchain (the qemu-system-riscv package) was part of the work.

Using ls, cat, echo, wc and a pipeline showed that xv6 provides the familiar Unix abstractions in a much smaller form, and reading user/cat.c, user/user.h and kernel/sysfile.c exposed the boundary between unprivileged programs and privileged system-call handlers.

Implementing user/sleep.c, registering it in UPROGS and testing valid and invalid calls demonstrated the whole path from a user program to a real system call in a kernel I compiled myself. Debugging the Makefile blank line and the sleep/pause rename also taught me to read build errors carefully and to check the headers of the actual source tree rather than relying on the handout alone.

REFERENCES

[1] Operating Systems Lab 2 handout. Meet the OS You Will Build. FCIM / FAF / UTM, 2026–2027.

[2] xv6-riscv source repository, https://github.com/mit-pdos/xv6-riscv.